```
!pip install -q pyspark
```

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .master("local[*]") \
    .appName("ColabSpark") \
    .getOrCreate()

print("Spark Version:", spark.version)
```

```
Spark Version: 4.0.2
```

```
from pyspark.sql.functions import col
```

```
df = spark.read.json('sample_books.json')
df.printSchema()
```

```
root
 |-- author: string (nullable = true)
 |-- edition: string (nullable = true)
 |-- price: double (nullable = true)
 |-- title: string (nullable = true)
 |-- year_written: long (nullable = true)
```

```
#1. Count number of records
df.count()
```

```
13
```

```
#2. Show title,price of the book.
df.select("title","price").show()
```

```
+--------------------+-----+
|               title|price|
+--------------------+-----+
|    Northanger Abbey| 18.2|
|       War and Peace| 12.7|
|       Anna Karenina| 13.5|
|       Mrs. Dalloway| 25.0|
|           The Hours|12.35|
|    Huckleberry Finn| 5.76|
|         Bleak House| 5.75|
|          Tom Sawyer| 7.75|
| A Room of One's Own| 29.0|
|         HarryPotter|19.95|
|One Hundred Years...| 14.0|
|Hamlet, Prince of...| 7.95|
|   Lord of the Rings|27.45|
+--------------------+-----+
```

```
#3. Show the records of books which have been written after 1950 and price of book is greater than 10.
books_df = df.filter((col("year_written") > 1950) & (col("price") > 10))
books_df.show()
```

```
+--------------------+-----------------+-----+--------------------+------------+
|              author|          edition|price|               title|year_written|
+--------------------+-----------------+-----+--------------------+------------+
|Cunnningham, Michael|   Harcourt Brace|12.35|           The Hours|        1999|
|       Rowling, J.K.|   Harcourt Brace|19.95|         HarryPotter|        2000|
|             Marquez|Harper  Perennial| 14.0|One Hundred Years...|        1967|
+--------------------+-----------------+-----+--------------------+------------+
```

```
#4. Show the title ,price and year of books of question 3
books_df.select("title", "price", "year_written").show()
```

```
+--------------------+-----+------------+
|               title|price|year_written|
+--------------------+-----+------------+
|           The Hours|12.35|        1999|
|         HarryPotter|19.95|        2000|
```

```
|One Hundred Years...| 14.0|        1967|
+--------------------+-----+------------+
```

```
#5. Count the number of question 4 records.
books_df.count()
```

```
3
```

```
#6. Show the title and year written of 'Harry Potter' Book.
df.filter(col("title").contains("Harry")).select("title", "year_written").show()
```

```
+-----------+------------+
|      title|year_written|
+-----------+------------+
|HarryPotter|        2000|
+-----------+------------+
```

```
from pyspark.sql.functions import col, mean
from pyspark.sql.functions import when
```

```
df = spark.read.json("people.json")
df.show()
```

```
+----+-------+
| age|   name|
+----+-------+
|NULL|Michael|
|  30|   Andy|
|  19| Justin|
|  29|   Mary|
|NULL|   John|
|  36|   Juhi|
|  13|Jasmeen|
|  12|  Seeta|
|  45|   NULL|
|  34|  Mohan|
|  18|  Kayen|
|  67|   NULL|
|  21|  Caley|
|  19|  Shyam|
+----+-------+
```

```
#1. Count number of records
df.count()
```

```
14
```

```
#2. create a new column with the simple copy of age column.
df1 = df.withColumn("age_copy", col("age"))
df1.show()
```

```
+----+-------+--------+
| age|   name|age_copy|
+----+-------+--------+
|NULL|Michael|    NULL|
|  30|   Andy|      30|
|  19| Justin|      19|
|  29|   Mary|      29|
|NULL|   John|    NULL|
|  36|   Juhi|      36|
|  13|Jasmeen|      13|
|  12|  Seeta|      12|
|  45|   NULL|      45|
|  34|  Mohan|      34|
|  18|  Kayen|      18|
|  67|   NULL|      67|
|  21|  Caley|      21|
|  19|  Shyam|      19|
+----+-------+--------+
```

```
#3. create a new column containing ages of people one year bigger.
df2 = df.withColumn("age_plus_one", col("age") + 1)
df2.show()
```

```
+----+-------+------------+
| age|   name|age_plus_one|
+----+-------+------------+
|NULL|Michael|        NULL|
|  30|   Andy|          31|
|  19| Justin|          20|
|  29|   Mary|          30|
|NULL|   John|        NULL|
|  36|   Juhi|          37|
|  13|Jasmeen|          14|
|  12|  Seeta|          13|
|  45|   NULL|          46|
|  34|  Mohan|          35|
|  18|  Kayen|          19|
|  67|   NULL|          68|
|  21|  Caley|          22|
|  19|  Shyam|          20|
+----+-------+------------+
```

```
#4. Show the records where age column is missing.
df3 = df.filter(col("age").isNull())
df3.show()
```

```
+----+-------+
| age|   name|
+----+-------+
|NULL|Michael|
|NULL|   John|
+----+-------+
```

```
#5. Fetch the records having atleast one non-null values.
df4 = df.na.drop(how="all")
df4.show()
```

```
+----+-------+
| age|   name|
+----+-------+
|NULL|Michael|
|  30|   Andy|
|  19| Justin|
|  29|   Mary|
|NULL|   John|
|  36|   Juhi|
|  13|Jasmeen|
|  12|  Seeta|
|  45|   NULL|
|  34|  Mohan|
|  18|  Kayen|
|  67|   NULL|
|  21|  Caley|
|  19|  Shyam|
+----+-------+
```

```
#6. Fill the missing values of age column with mean.
mean_age = df.select(mean("age")).collect()[0][0]
df5 = df.na.fill(mean_age, subset=["age"])
df5.show()
```

```
+---+-------+
|age|   name|
+---+-------+
| 28|Michael|
| 30|   Andy|
| 19| Justin|
| 29|   Mary|
| 28|   John|
| 36|   Juhi|
| 13|Jasmeen|
| 12|  Seeta|
| 45|   NULL|
| 34|  Mohan|
| 18|  Kayen|
| 67|   NULL|
```

```
| 21|  Caley|
| 19|  Shyam|
+---+-------+
```

```
#7. Write a sql query to find the people with age greater than 19.
df.createOrReplaceTempView("people")
spark.sql("SELECT * FROM people WHERE age > 19").show()
```

```
+---+-----+
|age| name|
+---+-----+
| 30| Andy|
| 29| Mary|
| 36| Juhi|
| 45| NULL|
| 34|Mohan|
| 67| NULL|
| 21|Caley|
+---+-----+
```